MINI-PROJECT REPORT

on

BLOOM: YOUR COLOR GUIDE

Submitted in partial fulfillment for the award of the Degree

of

Bachelor of Technology

in

Computer Science and Engineering

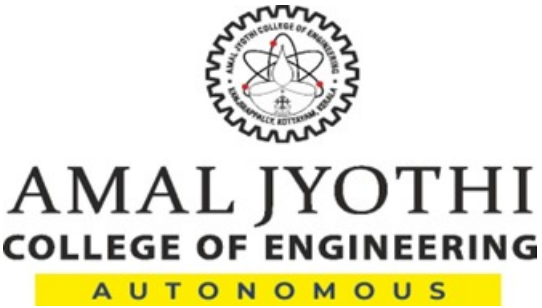
by

HRIDYA CATHERENE YESUDAS (AJC21CS080)
JOSE V JOSEPH (AJC21CS093)
MATHEW ABRAHAM (AJC21CS101)

under the guidance of

Ms. ANISHAMOL ABRAHAM

Assistant Professor



Department of Computer Science and Engineering

Amal Jyothi College of Engineering
Kanjirappally-686518

May 2024

DECLARATION

We undersigned hereby declare that the mini-project report **"Bloom: Your Color Guide"** submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Ms.Anishamol Abraham.This submission represents our ideas in our own words and where ideas or words of others have been included; we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place:Kanjirappally

Date:

Hridya Catherene Yesudas

Jose V Joseph

Mathew Abraham

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMAL JYOTHI COLLEGE OF ENGINEERING(AUTONOMOUS)
KANJIRAPPALLY



CERTIFICATE

This is to certify that the mini-project report entitled **"Bloom: Your Color Guide"** is a bonafide record of the mini project work carried out by **Hridya Catherene Yesudas (AJC21CS080), Jose V Joseph (AJC21CS093), Mathew Abraham (AJC21CS101)**, during the academic year 2023-2024 in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering by APJ Abdul Kalam Technological University. Certified further that the work reported herein does not form part of any other thesis or dissertation submitted to any other University or Institute for any purpose.

Ms. Anishamol Abraham
Project Guide

Dr. Juby Mathew
Head of the Department

EXTERNAL EXAMINER

INTERNAL EXAMINER

ACKNOWLEDGEMENT

First of all we sincerely thank the **Almighty GOD** who is most beneficent and merciful for giving us knowledge and courage to complete the project successfully.

We derive immense pleasure in expressing our sincere thanks to our Manager and Managing Trustee, **Very Rev. Fr. Boby Alex Mannamplackal, Rev. Fr. Dr. Roy Abraham Pazhayaparampil**(Director, Administration) and to our Principal, **Dr. Lillykutty Jacob** for the kind cooperation in all aspects of our project.

We owe a great depth of gratitude towards **Dr. Juby Mathew**, HOD, Department of Computer Science and Engineering for his kind co-operation in all aspects of our Project. We express our sincere thanks to our internal guide, **Ms. Anishamol Abraham**, Assistant Professor and our Project Co-ordinators **Ms. Niya Joseph**, Assistant Professor & **Ms. Elisabeth Thomas**, Assistant Professor for their encouragement and motivation during the project.

We are indebted to our beloved teachers for their cooperation and suggestion throughout the project which significantly helped us a lot. We also thank all our friends and classmates for their interest, dedication and encouragement shown towards the project. We convey hearty thanks to our parents for their moral support, suggestion and encouragement to make this venture a success.

ABSTRACT

In an effort to empower individuals with color blindness, this project proposes a web application that bridges the divide in color perception. The application utilizes real-time camera analysis, employing a central cursor to pinpoint specific colors within the user's field of view. Functionality extends beyond live camera use, allowing users to upload images and identify colors at designated points within them. This surpasses simple color naming by offering user-friendly descriptions that bridge the gap through relatable comparisons or real-world applications of the identified colors. To further enrich the user experience, the application generates complementary and similar color palettes based on the detected color. This comprehensive approach fosters a deeper understanding of color for users with color blindness, ultimately enhancing their ability to navigate and interact with the visual world in a more confident and meaningful way.

CONTENTS

ABSTRACT **ii**

1 Introduction **1**

1.1 Overview 1

1.2 Objective 1

1.3 Expected Outcome 2

2 Software requirements Specifications **3**

2.1 Introduction 3

2.1.1 Purpose 3

2.1.2 Intended Audience 3

2.1.3 Intended Use 3

2.1.4 Scope 4

2.1.5 Definitions and Acronyms Used 4

2.2 Overall Description 5

2.2.1 User Needs 5

2.3 System Features and Requirements 5

2.3.1 Functional Requirements 5

2.3.2 External Interface Requirements 7

2.4 System Features 7

2.4.1 Non-Functional requirements 9

2.5 Use Case Diagram 11

2.6 Activity Diagram 12

3 Software Adopted **14**

3.1 Introduction 14

3.1.1 Front-end Development 14

3.1.2 Image Processing 15

3.1.3 Back-end Development 15

3.1.4 Data Hosting 15

3.1.5 Ishihara Color Blindness Test 15

3.2 Excel 16

3.3 VSCODE for Python Programming Language 16

3.4 Flask 16

3.5 Color Recognition and Processing Libraries 17

3.5.1 Real-Time Color Detection 17

3.5.2 Image Color Analysis 17

3.5.3 Color Blindness Test Integration 17

4 The Proposed System **18**

4.1 Existing Systems 18

4.1.1 Color Blind Pal 18

4.1.2 Chromatic Vision Simulator 19

4.1.3 Color Binoculars 19

4.2 Ishihara Test 19

4.3 Color Detection Module 20

4.3.1 Image Upload 20

4.3.2 Live Camera 20

4.4 System Workflow 20

4.4.1 User Initialization 20

4.4.2 Color Detection 21

4.4.3 Image Upload and Color Detection 21

4.4.4 Live Camera Color Detection 21

4.4.5 System Architecture 21

5 Implementation **23**

5.1 User Interface Development 23

5.2 Back–end Development 27

5.2.1 Flask Application 27

5.2.2 Data Processing 27

5.3 Color Detection Using OpenCV 28

5.4 Testing and Validation 28

6 Conclusion **30**

Bibliography **32**

A Sample Code **33**

LIST OF FIGURES

2.1 Use case Diagram 11

2.2 Activity Diagram 12

4.1 System Architecture 22

5.1 Loading Page 24

5.2 Menu page 24

5.3 Ishihara Test 25

5.4 Home Page 25

5.5 Color detection through image upload 26

5.6 Color detection through live camera 26

CHAPTER 1

INTRODUCTION

1.1 Overview

This web application aids color-blind individuals in identifying colors. It uses the device camera for real-time color detection with a central cursor and allows image uploads for color analysis. The application provides descriptions of colors, their real-life applications, and lists of contrasting and similar colors.

1.2 Objective

According to the latest studies,it is estimated that around 300 million people worldwide have color blindness.In men, the prevalence of color blindness is around 5.0% to 8.0%, while in women the prevalence is only 0.5% to 1.0%. The lack of efficient applications that aid the day-to-day activities of a colorblind person in the market pays to the significance of the web application,'Bloom'. The primary objective of this web application is to empower color-blind individuals by providing them with a reliable tool for color detection and understanding. By utilizing the device camera for live color identification and enabling image uploads for analysis, the application aims to offer a comprehensive solution for recognizing and interpreting colors in everyday life. The descriptions and color lists are intended to enrich the user's knowledge and assist in making informed decisions related to color. [1]

1.3 Expected Outcome

The expected outcome of our web application, Bloom:Your Color Guide is to empower individuals affected by colorblindness.Users of this application can expect to gain accurate and immediate information about colors in their surroundings and in images providing enhanced comprehension of colors through relatable comparisons and tangible real-life applications . The live color detection feature, coupled with detailed descriptions and lists of contrasting and similar colors, will help users to better understand and differentiate between colors. This improved color recognition will aid in various aspects of daily life, from selecting clothing and home decor to understanding visual information in media, fostering better color comprehension and interaction, ultimately enhancing the quality of life for color-blind individuals.

CHAPTER 2

SOFTWARE REQUIREMENTS SPECIFICATIONS

2.1 Introduction

2.1.1 Purpose

The purpose of this project report is to detail the development and functionalities of a web application designed specifically for colorblind individuals. The application aims to assist colorblind users in identifying colors in real-time through live color detection using the device's camera and by analyzing images uploaded from the user's device. On detecting the color,the application provides descriptions of the detected colors using real-life applications or comparisons, and it also offers lists of contrasting and similar colors for each detected color.In addition to the color detection,the application provides a free Ishihara Test, test for detection of color vision deficiencies,to help the user to detect the condition they possess.

2.1.2 Intended Audience

This document is intended for both the users and developers of the application and will be proposed to the faculty and guide for its approval.

2.1.3 Intended Use

This document presents a detailed examination of the application's features, elucidating both the formal and informal requirements to establish a comprehensive context for the technical

specifications.

2.1.4 Scope

The scope of the web application for color-blind individuals focuses on enhancing color recognition and understanding. It features live color detection using the device camera, with a central cursor to identify specific colors. Users can also upload images and tap on any point to detect the color at that spot. The application provides detailed descriptions of each color, explaining their real-life applications and similarities to more recognizable colors. Additionally, it offers lists of contrasting and similar colors to aid in color differentiation and comparison. This tool aims to improve accessibility and enrich the visual experiences of color-blind users in their daily activities. [2]

2.1.5 Definitions and Acronyms Used

HTML (Hypertext Markup Language) is a prevalent markup language used for creating and structuring web pages. It defines the elements and layout of a web page, providing a standardized format for organizing text, images, links, and other media, which allows for the development of interactive and visually appealing websites.

CSS (Cascading Style Sheets) is responsible for the visual presentation of web experiences, CSS allows for precise control over the appearance of web pages. It dictates layout, fonts, colors, and other visual elements, ensuring a consistent and aesthetically pleasing user interface.
JS(JavaScript) is a dynamic, high-level programming language that is a core technology of the World Wide Web which enables the creation of interactive and responsive web pages by allowing developers to implement complex features such as content updates, multimedia controls, animated graphics, and more. JavaScript runs on the client side, meaning it executes directly in the user's browser, leading to faster and more efficient user experiences without the need for server-side processing.

Excel,a popular spreadsheet application developed by Microsoft, is widely used for data

cleaning. This process involves identifying and correcting errors, inconsistencies, and inaccuracies in datasets to ensure data quality and reliability.

Hex code , short for hexadecimal code, is a way to represent colors using a combination of hexadecimal digits (0-9 and A-F). Each digit represents a specific value in the red, green, and blue (RGB) color model, the most common system for displaying colors on electronic devices.

2.2 Overall Description

2.2.1 User Needs

The web application for color-blind individuals provides enhanced color recognition through live detection via device camera, image upload capability, and detailed color descriptions. Users seek assistance in understanding colors' real-life applications and distinguishing between contrasting and similar hues.

2.3 System Features and Requirements

2.3.1 Functional Requirements

1. Color Detection

The application provides two modes for color identification. In Live Camera Mode, users can access the device camera to display a live video feed with an overlaid central cursor. When the user taps or clicks, the app captures the color beneath the cursor, allowing for real-time color detection. In Image Upload Mode, users can upload images from their device and then tap or click on a specific point within the image to identify the color at that location. This combined functionality enables users to determine colors either directly from a live camera feed or from images stored on their device.

2.Color Information

The Color Information feature provides detailed insights into detected colors. It includes the technical name of the detected color and offers user-friendly descriptions using real-life applica-

tion examples, such as ”fire engine red,” and similarities to well-known objects, like ”sky blue.” This feature ensures users can understand and identify colors accurately and intuitively.

3.Color Matching and Contrast

The Color Matching and Contrast feature enhances the user's ability to work with colors by generating a list of colors that offer high contrast to the identified color, ensuring excellent visibility and aesthetic appeal. Additionally, it provides a list of colors similar to the identified one, broadening the user's color vocabulary and enabling more nuanced and informed color choices.

4.Color Blindness test

The color Blindness test feature integrates the Ishihara Color Blindness test to accurately assess the type of color blindness a user may have. It presents the test results in a clear and understandable format, indicating both the type and severity of color blindness detected. This functionality is crucial for users, as it helps them understand their color perception limitations and enables them to make informed decisions about color use and accessibility in various applications and environments.

5.User Interface

The User Interface (UI) of the application is meticulously designed to prioritize user-friendliness and intuitiveness. It ensures accessibility for users with different levels of color blindness by adhering to high contrast design principles and incorporating alternative methods of information presentation such as icons and text descriptions. This approach not only enhances usability but also accommodates diverse user needs, making the application accessible and effective for all individuals, regardless of their color perception abilities.

6.Back-end

The system features a robust back-end implemented with a database system that securely stores and retrieves essential data for color detection and descriptions. This database is designed to ensure both security and scalability, capable of handling substantial volumes of data efficiently as the application grows. By leveraging a secure database architecture, the system safeguards sensitive information related to color detection results and user interactions. This setup not only supports current operational needs but also positions the application for future expansion and

increased data demands, maintaining reliability and performance at all times. [3]

2.3.2 External Interface Requirements

External interface requirements describe the interactions and interfaces your system has with external entities. Here are some external interface requirements for your project:

1. User Inputs - The web application possess provision for users to export an image from their device storage and also to access their camera to provide live detection The system should validate and handle user inputs effectively to ensure accurate and reliable results.

2.4 System Features

System Functionality for Color Detection Web Application This web application addresses the needs of color blind individuals by offering a robust set of functionalities designed to enhance their color perception and interaction with the visual world.

1.Color Identification

The Color Identification feature of the application offers two primary functionalities. First, it integrates real-time camera capabilities, utilizing the device's camera with a central cursor for precise targeting of specific colors within the user's surroundings. This allows users to identify colors in real-time by simply pointing the camera and interacting with the screen. Second, the application supports image upload, enabling users to upload images from their device. Within uploaded images, users can identify colors by tapping on any desired point, facilitating accurate color detection within stored photos or images. These combined features provide versatile tools for users to identify and work with colors effectively in both immediate and stored visual contexts.

2. Comprehensive Color Information

The Comprehensive Color Information feature of the application offers users detailed insights into detected colors through two key functionalities. Firstly, it accurately identifies and displays the hexadecimal (hex) code of the detected color, providing precise technical information. Sec-

ondly, the application goes beyond technical terms by offering user-centric color descriptions. These descriptions are tailored to be clear and informative, particularly for color blind users, and include relatable comparisons to everyday objects such as ”sky blue” or ”lime green.” Additionally, the application provides real-world applications of the color, like ”commonly used for stop signs,” enhancing users' understanding and practical use of identified colors across various contexts. This comprehensive approach ensures that all users, regardless of color perception abilities, can effectively utilize and appreciate the color information provided by the application.

3. Advanced Color Exploration Tools

The application includes a Contrastive Color Recommendation Engine designed to enhance visual contrast by generating a list of colors that offer strong visual differentiation from the identified color. This feature is particularly useful in design contexts where achieving clear readability and aesthetic appeal is crucial, as well as in promoting accessibility considerations for users with varying visual capabilities. Additionally, the application dynamically generates a Similar Color Palette, which expands the user's color vocabulary by presenting a range of colors closely related to the identified color. This functionality aids users in discerning subtle variations within the color spectrum, which can be challenging to perceive due to color blindness or other visual impairments. Together, these features provide comprehensive tools for users to effectively manage and explore color options, ensuring both usability and inclusivity in color-related tasks.

4. Accessibility Enhancements

The application offers an optional Ishihara Color Blindness Test, enabling users to identify the specific type of color blindness they experience. Additionally, the application features an intuitive user interface design that prioritizes user-friendliness and accessibility, catering to a diverse range of visual abilities. This combination of functionality and design ensures that the application is both informative and easy to use for all users, regardless of their visual capabilities.

2.4.1 Non-Functional requirements

1.Usability and Accessibility

The application features an intuitive and user-friendly interface, ensuring ease of use for all users. It adheres to WCAG (Web Content Accessibility Guidelines) standards, making it accessible to color-blind users. Additionally, font sizes, color contrasts, and navigation elements are optimized for readability to enhance user experience.

2.Performance

The application ensures responsive and efficient real-time color detection using the device camera. It also handles image uploads promptly, even when dealing with large files, maintaining smooth and uninterrupted functionality.

3.Accuracy of Color Detection

The cursor's position is highly accurate in live color detection, providing precise color information based on the user's tap or cursor position. This ensures that users receive reliable and exact color details.

4.Security and Privacy

User-uploaded images are securely processed, addressing privacy concerns related to camera usage. The application is designed to respect user privacy and does not collect unnecessary personal data, ensuring a secure and trustworthy user experience.

5.Compatibility

The application is designed to work seamlessly across different devices, including desktops, tablets, and mobiles. It is also compatible with all major browsers, ensuring a consistent user experience regardless of the platform.

6.Performance Across Lighting Conditions

The color detection algorithm is optimized to perform consistently under various lighting conditions, such as bright light, low light, and artificial light, ensuring accurate color detection in diverse environments.

7.Descriptive Color Information

The application provides concise, informative, and relatable color descriptions. Real-life appli-

cations of colors are accurately represented, offering helpful insights for users.

8.Ishihara Color Blind Test

The Ishihara test feature accurately identifies the type of color blindness a user may have. The test is designed to be easy to understand and interpret, providing clear results for users.

9.Scalability and Maintenance

The application is scalable to accommodate future updates and enhancements. Regular maintenance and bug fixes are performed to ensure a reliable user experience, adapting to evolving user needs and technological advancements.

10.Performance Metrics

Well-defined performance metrics, such as response time and image processing time, are established and maintained to meet acceptable thresholds. These metrics ensure the application performs efficiently and meets user expectations.

2.5 Use Case Diagram

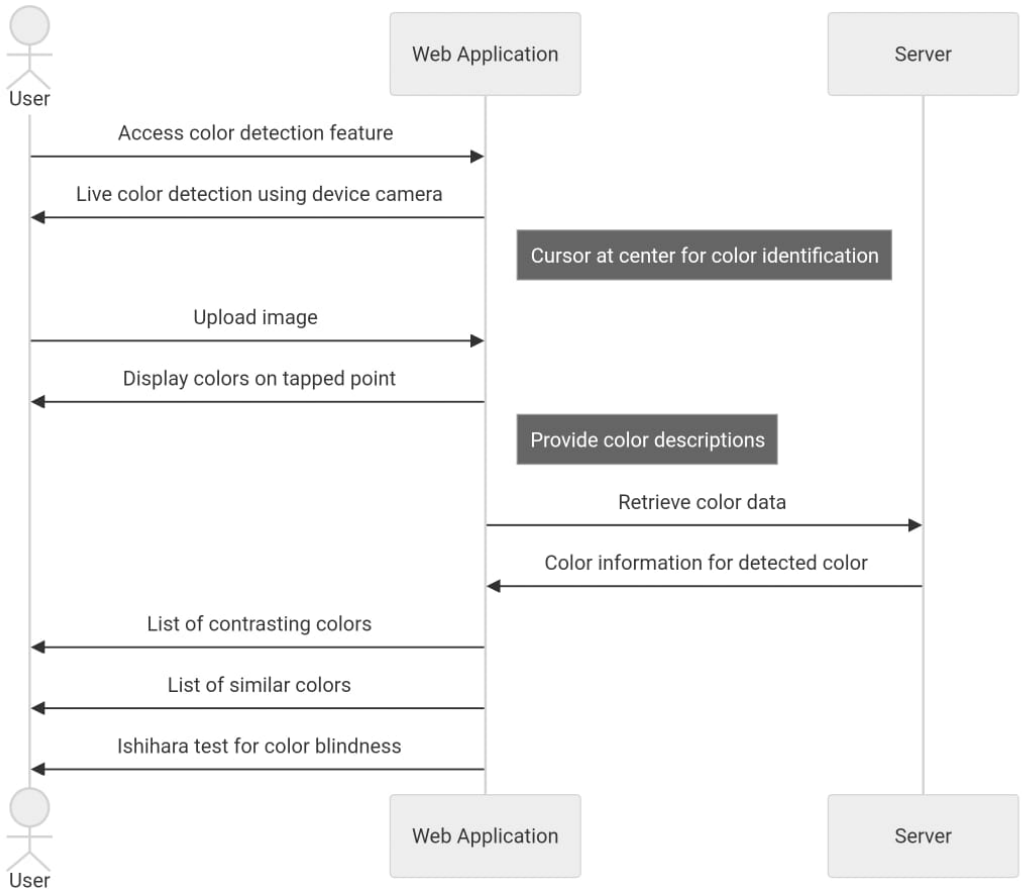


Figure 2.1: Use case Diagram

From the Figure 2.1 we can understand that system utilizes a device camera to capture a live image. This image is then uploaded to a server for processing. The server transmits the processed image back to the user's web application. The user can then select a point on the image, and the server will provide color information for that specific point. In addition, the system can generate lists of similar and contrasting colors. The user can also perform an Ishihara test for color blindness through the web application.

2.6 Activity Diagram

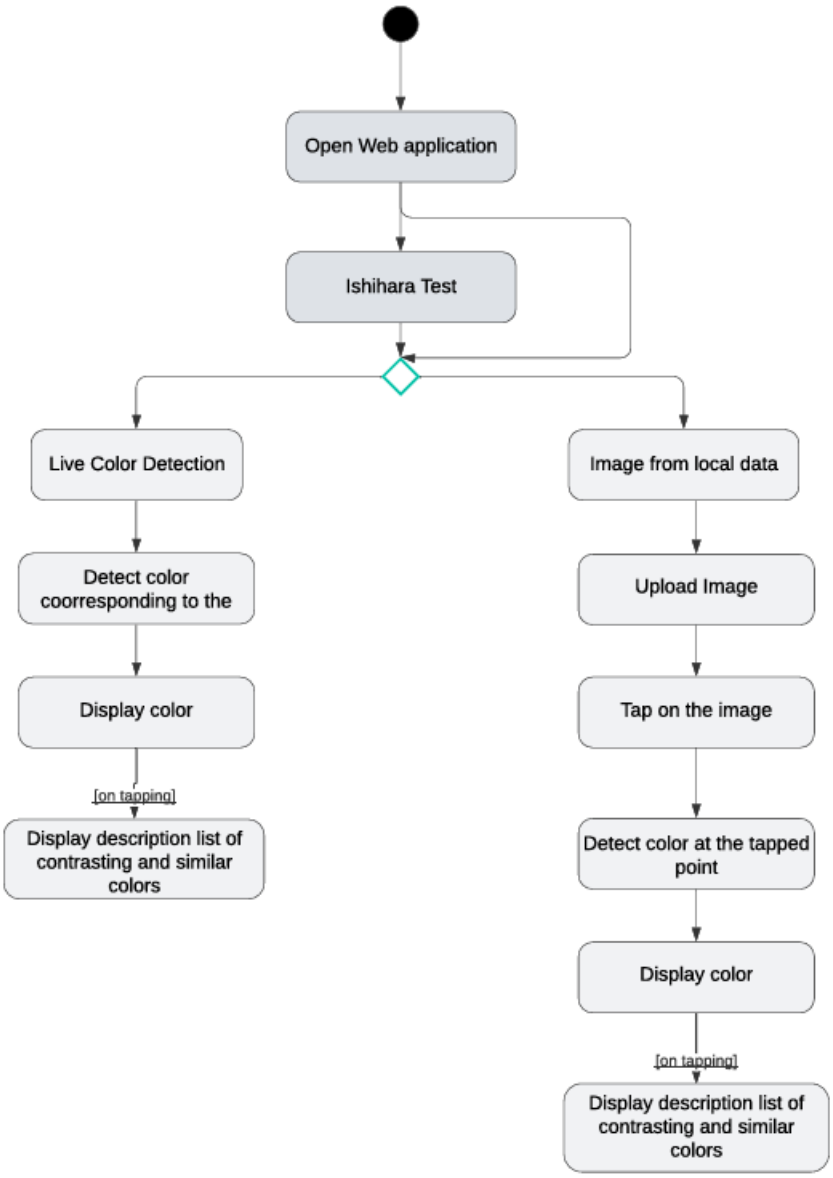


Figure 2.2: Activity Diagram

The flowchart outlines the workflow of a web application designed for live color detection. The process commences with the user launching the web application. Subsequently, the user

encounters two options: live color detection or image analysis using local data. If the user opts for live color detection, the device camera is activated to capture an image. The captured image is then uploaded to a server for color analysis. The server transmits the processed data back to the web application, which displays the identified colors. The user has the ability to tap on a specific region of the image for further examination. Upon tapping the image, the server ascertains the color corresponding to the selected area and the web application presents a list of colors similar to and contrasting with the chosen color.

Alternatively, the user can elect to analyze an image stored on their device. In this scenario, the user uploads the local image, and the process proceeds through the same steps (5 through 9) as with live color detection. The key distinction is that the system analyzes the uploaded image from local data rather than a live image captured from the device camera.

CHAPTER 3

SOFTWARE ADOPTED

3.1 Introduction

This report outlines the software landscape envisioned for a web application designed to support individuals with color blindness.

3.1.1 Front-end Development

HTML

For structuring the web pages and content.

CSS

For styling and designing the user interface to ensure a visually appealing and user-friendly experience.

JavaScript

For styling and designing the user interface to ensure a visually appealing and user-friendly experience.

3.1.2 Image Processing

OpenCV (Open Source Computer Vision Library)

Utilized for image processing and analysis, enabling color detection and extraction of color information from user-uploaded images. [6]

3.1.3 Back-end Development

OpenCV Python with Flask Framework

The back-end of the application is developed using Python programming language with the Flask web framework. Flask is chosen for its simplicity and flexibility in building web applications. Python provides a wide range of libraries and tools suitable for image processing and data handling, which are essential for color detection and analysis.

3.1.4 Data Hosting

Server Hosting

The application data is hosted on a server. This server provides the infrastructure necessary to store and manage the data used by the application. It ensures that the application can access and process data efficiently.

3.1.5 Ishihara Color Blindness Test

JavaScript Implementation

To incorporate the Ishihara color blindness test within the web application, allowing users to determine the type of color blindness they have.

3.2 Excel

Microsoft Excel, a widely used spreadsheet application, offers a range of powerful features and functionalities that make it an excellent choice for data cleaning tasks. This report explores the various ways Excel can be utilized to clean and prepare data for analysis

3.3 VSCODE for Python Programming Language

Python, a high-level, general-purpose programming language, is at the heart of the virtual assistant project. Renowned for its readability and simplicity, Python facilitates efficient and expressive coding. Its vast standard library and extensive community support make it an ideal choice for developing diverse functionalities in the virtual assistant. Python's versatility allows seamless integration of components, ranging from speech recognition to natural language processing. The language's syntax promotes clean and maintainable code, crucial for a project with intricate functionalities. Overall, Python's adaptability, rich ecosystem, and user-friendly syntax contribute significantly to the success of the virtual assistant, ensuring robustness and ease of development

3.4 Flask

Flask, a lightweight web framework for Python, to seamlessly integrate the front-end and back-end components. Flask provides a simple yet powerful structure for building web applications, making it an ideal choice for our project's needs. With Flask, we were able to create APIs to facilitate communication between the front-end user interface and the back-end functionalities of the application. Its flexibility allowed us to efficiently handle HTTP requests and responses, enabling smooth interactions between the user and the virtual assistant. By leveraging Flask, we ensured a cohesive and intuitive user experience, while also maintaining scalability and ease of maintenance for future updates and enhancements. [7]

3.5 Color Recognition and Processing Libraries

This web application incorporates OpenCV, a powerful open-source computer vision library, to address the needs of color blind users. Here's how OpenCV contributes to the project:

3.5.1 Real-Time Color Detection

OpenCV's image processing capabilities enable the application to analyze the live camera feed. By focusing on the central cursor position, OpenCV extracts the relevant pixel data and performs color space conversions (e.g., BGR to HSV) to facilitate accurate color identification.

3.5.2 Image Color Analysis

Uploading images allows users to analyze pre-captured content. OpenCV facilitates loading the image file and identifying the color at the user's tap location using similar techniques employed in real-time detection.

3.5.3 Color Blindness Test Integration

While not directly a part of OpenCV, the application can leverage existing libraries or web services to incorporate the Ishihara color blindness test. This functionality can be integrated using OpenCV's image processing capabilities to analyze test plates and provide results.

CHAPTER 4

THE PROPOSED SYSTEM

The Bloom application consists of several key components designed to provide accurate color identification and a user-friendly experience.

4.1 Existing Systems

4.1.1 Color Blind Pal

Color Blind Pal is a mobile application available on both Android and iOS platforms, designed to assist color-blind individuals in identifying colors and distinguishing between different hues. The application offers various features, such as a real-time color identifier that uses the device's camera to detect colors and provide users with the names and descriptions of the colors they are viewing. Additionally, Color Blind Pal includes a color filter option that allows users to adjust the colors displayed on their screen to enhance contrast and visibility. However, one of the main limitations of Color Blind Pal is its dependency on mobile devices, which restricts its usability for individuals who prefer or require a web-based solution. Furthermore, the application may not provide comprehensive information on the real-life applications of the detected colors or offer a detailed list of contrasting and similar colors. The interface may also not be as user-friendly for all individuals, particularly those who are not tech-savvy or have difficulties navigating mobile applications.

4.1.2 Chromatic Vision Simulator

Chromatic Vision Simulator is another mobile application designed to help color-blind individuals by simulating different types of color vision deficiencies. Available on both Android and iOS platforms, the application allows users to view their surroundings or images through the lens of various color blindness simulations. However, one of the primary limitations of Chromatic Vision Simulator is its focus on simulating color vision deficiencies rather than providing real-time color detection and identification. Additionally, the application may not offer detailed descriptions of detected colors, their real-life applications, or lists of contrasting and similar colors. Similar to Color Blind Pal, its dependency on mobile devices may restrict its usability for individuals who prefer a web-based solution.

4.1.3 Color Binoculars

Color Binoculars is a mobile application developed by Microsoft, available on iOS devices, designed to assist color-blind individuals in distinguishing between colors by applying filters to enhance contrast. The application uses the device's camera to provide real-time color adjustments, making it easier for users to differentiate between various hues. However, a significant limitation of Color Binoculars is its availability exclusively on iOS devices, which limits its accessibility for Android users or those who prefer a web-based solution. Additionally, the application may not provide comprehensive information on the real-life applications of detected colors or offer detailed lists of contrasting and similar colors. Like other similar applications, its dependency on mobile devices may restrict its usability for individuals who prefer or require a web-based solution.

4.2 Ishihara Test

The Ishihara test is a well-established method for diagnosing color blindness. Bloom includes a series of Ishihara plates to help users identify their type of color blindness. The test presents a series of images with numbers or patterns embedded in colored dots, which vary based on

the type of color vision deficiency. By analyzing the user's responses, Bloom can determine whether the user has monochromatic, protanopia, deuteranopia, or tritanopia color blindness. [4]

4.3 Color Detection Module

4.3.1 Image Upload

Users can upload images from their device to the application.Once an image is uploaded, it is displayed on the screen, and users can click on any part of the image to detect the color at that point. The application identifies the color and provides detailed information, including its description, hex code, similar colors, and contrasting colors.

4.3.2 Live Camera

For real-time color detection, Bloom utilizes the device's camera.The live camera feed is displayed on the screen, with a cursor indicating the point of color detection. The application continuously identifies the color at the cursor's position and provides the same detailed information as in the image upload method.

4.4 System Workflow

4.4.1 User Initialization

Upon launching the application, users are prompted to select their known type of color blindness or choose to take the Ishihara test to determine it. If the user opts to take the Ishihara test, they are guided through a series of images, and their responses are analyzed to identify their type of color blindness. The Ishihara test is displayed exclusively to new users, while existing users are directly presented with a color detection menu based on cookie data.

4.4.2 Color Detection

Based on the initial choice, users are directed to the appropriate color detection method. If the user selects a type of color blindness, they are taken directly to the color detection module, which includes options for image upload or live camera. If the user takes the Ishihara test, they are redirected to the color detection module after the test results are processed.

4.4.3 Image Upload and Color Detection

Users select and upload an image, which is then displayed within the application. They can click on any part of the image to detect the color at that point. The application identifies the color and displays detailed information about it, including a description, hex code, similar colors, and contrasting colors.

4.4.4 Live Camera Color Detection

When users enable the live camera feed, the application displays a real-time video feed from the device's camera with a cursor overlaid to indicate the detection point. As the cursor moves, the application continuously detects the color at its position and provides detailed information about it. This includes descriptive information, the hex code representation of the color, and potentially similar or contrasting colors for context. Users can interact with the feed by tapping or clicking to capture specific colors of interest, making the feature versatile for real-time color identification needs.

4.4.5 System Architecture

The system architecture (Figure 4.1) for the web application designed for color detection and analysis, including an Ishihara test for color blindness, consists of three main components: the front-end, back-end, and a CSV file. The front-end, built with HTML, CSS, and JavaScript, provides the user interface, enabling users to upload images or use a live camera feed. It also displays detected colors and their details and facilitates the Ishihara test. The back-end,

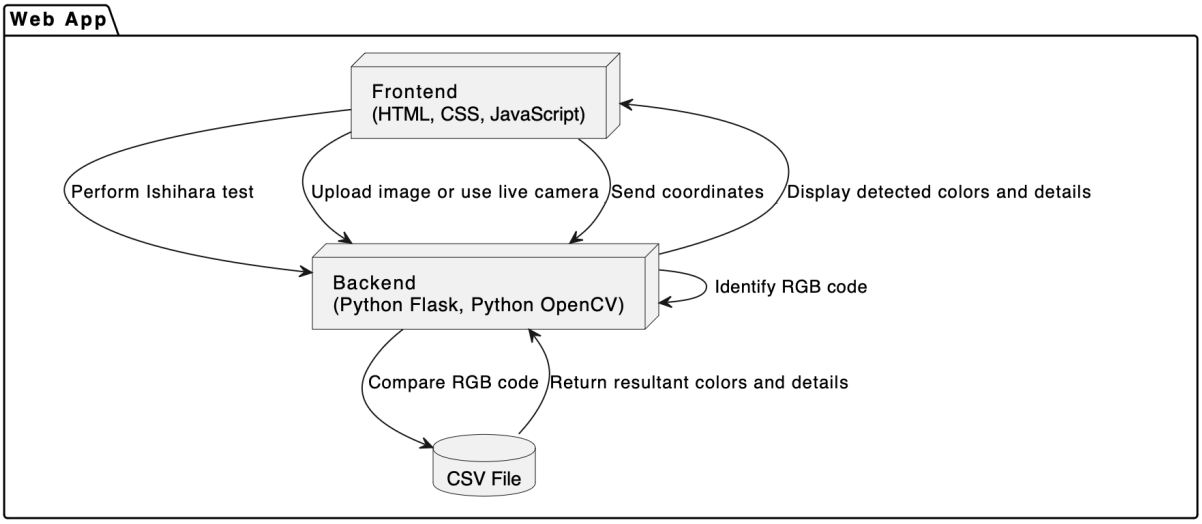


Figure 4.1: System Architecture

developed using Python Flask and Python OpenCV, processes the data received from the front-end. It identifies the RGB codes of the detected colors using OpenCV and compares these codes with a predefined set of colors stored in a CSV file. The CSV file contains the predefined colors and their corresponding RGB codes, which the back-end uses for comparison .

The workflow begins with the user performing the Ishihara test through the front-end. The user can then upload an image or use the live camera feature, with the front-end sending the image or video data and any user-specified coordinates to the back-end. The back-end processes this data to identify the RGB codes of the colors present. These codes are then compared with the predefined set of colors in the CSV file, allowing the back-end to identify the exact colors. The back-end returns the detected colors and their details to the front-end, which then displays the results to the user. This architecture ensures efficient color detection and analysis, providing detailed results, including conducting an Ishihara test for color blindness.

CHAPTER 5

IMPLEMENTATION

The implementation of Bloom involves several critical steps, integrating various technologies and ensuring a smooth user experience:

5.1 User Interface Development

The front-end of Bloom is designed to be user-friendly and accessible. The development involves:

1. **HTML:** Structuring the content and layout of the web application.

2. **CSS:**Styling the application to make it visually appealing and responsive across different devices.

3. **JavaScript:**The application utilizes JavaScript to enhance interactivity across its various pages. On the Home Page, users are introduced to the application's purpose and presented with options to select their known type of color blindness or to take the Ishihara test for diagnosis. The Ishihara Test Page features a series of images designed to diagnose different types of color blindness, accompanied by navigation controls for moving between images and submitting answers.

   Transitioning to the Color Detection Page, users are provided with options for either uploading images or activating the live camera detection feature. The interface dynamically displays the uploaded image or the live camera feed, enabling users to click on specific

points to detect colors in real-time. The application then presents detailed color detection results, which include a description of the color, its hexadecimal code, as well as lists of similar and contrasting colors for contextual understanding. This interactive approach ensures a user-friendly experience, facilitating seamless navigation and dynamic content updates based on user inputs and actions throughout the application.

UI Screenshots

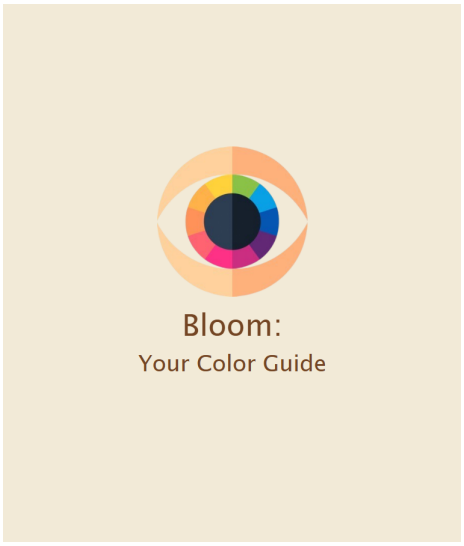


Figure 5.1: Loading Page

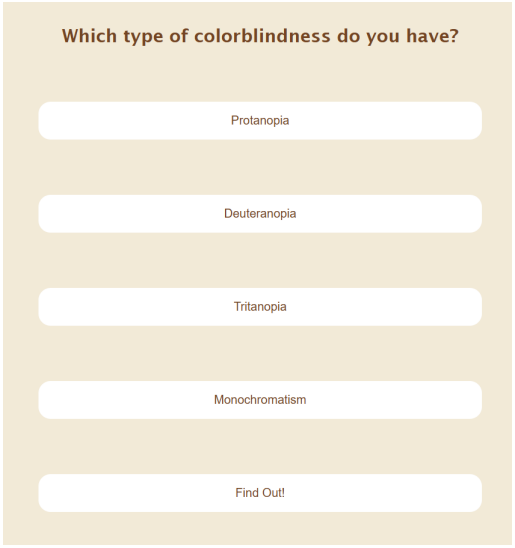
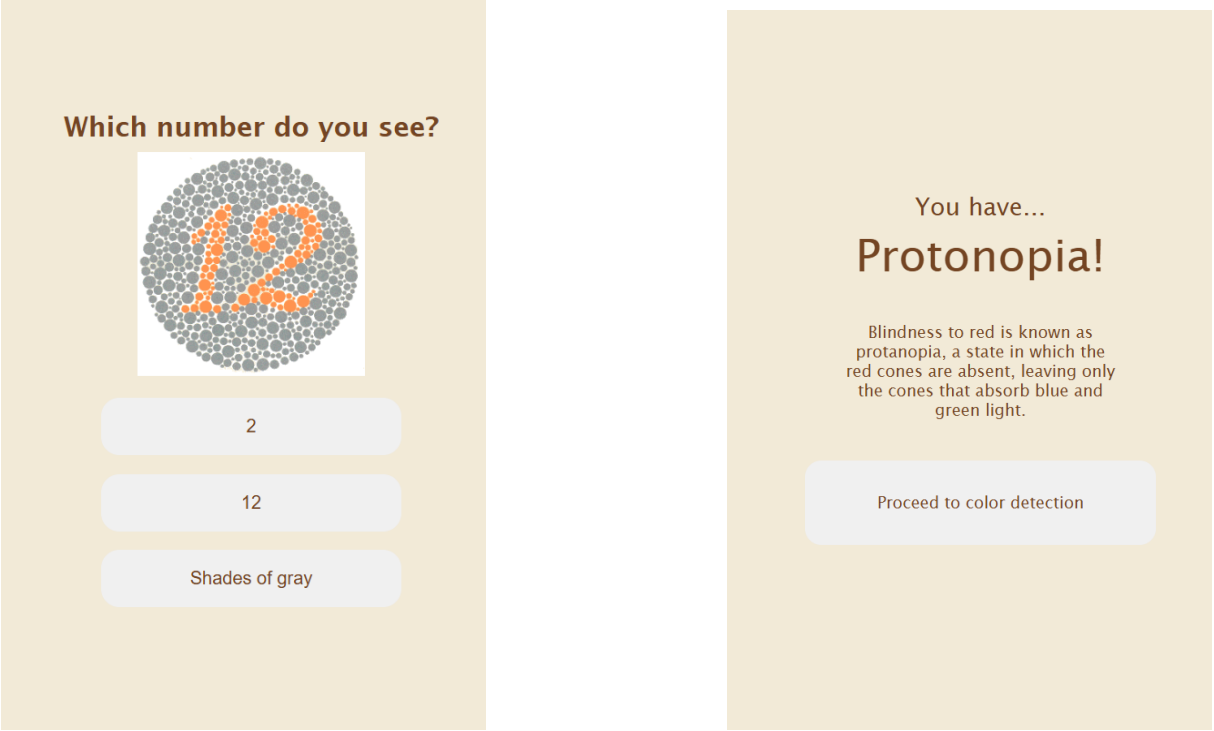


Figure 5.2: Menu page



(a) Quiz (b) Result

Figure 5.3: Ishihara Test

The menu page (Figure 5.2) allows users to select their type of color blindness. If users are unsure about their condition, they can take the Ishihara test (Figure 5.3). These pages are displayed only for first-time users, as the web application uses cookies to identify new visitors.

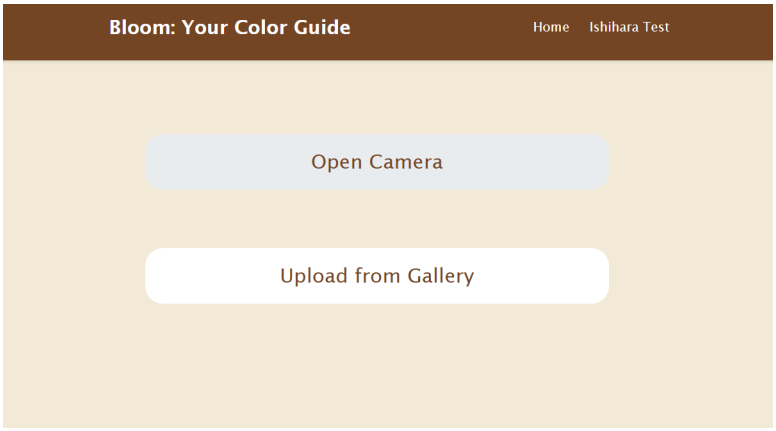


Figure 5.4: Home Page

From the home page (Figure 5.4), users can choose to open their device's camera to detect surrounding colors or upload an image from local storage for color detection.



Figure 5.5: Color detection through image upload

When an uploaded image is clicked or tapped, a pop-up appears showing the detected color name. A 'show more' button provides additional information, including a description of the color, similar colors, contrasting colors, and the hex code (Figure 5.5).

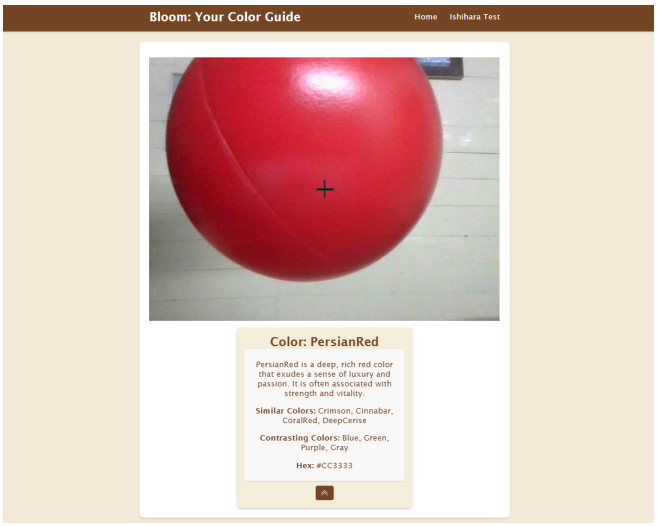


Figure 5.6: Color detection through live camera

When the 'Open Camera' option from home page is selected, the user's device camera will open and start streaming the output on the web page. A fixed pointer at the center of the camera frame acts as an indicator for detecting the color at that point (Figure 5.6).

5.2 Back–end Development

The back-end of Bloom is developed using Python and Flask, a lightweight web framework that provides the necessary functionalities to manage user interactions and data processing. The back-end development involves:

5.2.1 Flask Application

The application utilizes Flask to set up a server and configure routes that handle various user requests seamlessly. This includes routes for managing user sessions to ensure continuity throughout processes such as the Ishihara test and color detection. Flask handles form submissions, image uploads, and live camera feeds efficiently, ensuring smooth interaction between the application and the user. By leveraging Flask's capabilities, the application provides a robust back-end infrastructure that supports dynamic content updates and responsive handling of user inputs across different functionalities. This setup enhances overall usability and performance, making interactions with the application intuitive and efficient.

5.2.2 Data Processing

The back-end of the application performs critical functions to enhance user experience and functionality. Firstly, it analyzes user responses to the Ishihara test to accurately determine the type of color blindness present. This process involves evaluating user inputs and applying algorithms to interpret the results effectively. Secondly, the back-end utilizes OpenCV (Open Source Computer Vision Library) to process uploaded images and live camera feeds, enabling precise color detection. OpenCV's capabilities allow the application to analyze pixel data and identify colors accurately based on user interactions. [5]

Throughout these processes, the back-end ensures secure and efficient handling of user data and interactions, prioritizing data privacy and providing a seamless experience for users interacting with the application's color detection features. The back-end ensures secure and efficient handling of user data and interactions, providing a seamless experience for the users.

5.3 Color Detection Using OpenCV

1. **OpenCV,**a powerful library for computer vision, is used to implement the color detection functionality in Bloom. The implementation includes:

   (a) **Image Upload Processing**

      i. Users can upload an image to the application.

      ii. The image is processed using OpenCV to detect colors at specific points clicked by the user.

      iii. The detected color information, including description, hex code, similar colors, and contrasting colors, is displayed to the user.

   (b) **Live Camera Processing**

      i. The application accesses the device's camera and displays the live feed.

      ii. A cursor indicates the point of color detection, and the application continuously processes the frames to detect the color at the cursor's position.

      iii. The detected color information is displayed in real-time, providing immediate feedback to the user.

5.4 Testing and Validation

Thorough testing and validation are conducted to ensure the reliability and accuracy of the Bloom application. The testing process includes:

- **Unit Testing:**Individual components of the application are tested to ensure they function correctly in isolation. This includes testing the Ishihara test processing, image upload functionality, and live camera color detection.

- **Integration Testing:**The interactions between different components are tested to ensure they work seamlessly together. This includes testing the entire workflow from user initialization, through color detection, to the display of results.

- **User Testing:** Real users, particularly those with color blindness, are invited to test the application. Their feedback is used to make improvements and ensure the application meets their needs effectively.

- **Performance Testing:**The application's performance is tested to ensure it can handle multiple users and large image uploads efficiently. The live camera feed is tested for real-time color detection accuracy and responsiveness.

CHAPTER 6

CONCLUSION

Bloom is a sophisticated web application designed to assist individuals with color blindness by providing accurate color identification capabilities. It integrates intuitive user interfaces with robust back-end frameworks and advanced image processing techniques, ensuring a reliable solution for users with color vision deficiencies. By leveraging technologies like OpenCV and Flask, Bloom enables precise and real-time color detection across a range of practical scenarios, thereby enhancing users' ability to perform daily tasks with greater ease and confidence.

Bloom's development roadmap is all about enhancing user experience and functionality with innovative features. One key addition is Augmented Reality (AR) technology. This will let users use their device's camera to overlay color information directly onto real-world objects. This makes it easier to identify and differentiate colors in everyday situations, like choosing the right color while shopping, understanding traffic signals, and interpreting color-coded information. Bloom can also get smarter with machine learning. These algorithms will learn from user interactions, improving color identification accuracy over time. This means more precise color detection, personalized experiences, and better performance in complex images and challenging lighting conditions.For low-light situations, Bloom will incorporate a flashlight feature. This ensures consistent color detection across various lighting environments, eliminating inconsistencies caused by external light sources. Personalization is a big focus for Bloom. With user profiles, individuals can save their test results, detected colors, and preferences. This makes it easy to access past data, review previous interactions, and customize settings to suit personal

needs. Bloom may also expanding its accessibility by developing a mobile app for both Android and iOS platforms. This will allow users to take advantage of Bloom's features on the go, making color detection and exploration more convenient in any setting. Additionally, Bloom can further improve user experience through features such as voice assistance for auditory color descriptions, customization options for interface personalization. Finally, Bloom may build a collaborative user community. The app will include a platform for users to share their experiences, provide feedback, and suggest improvements. This two-way communication will help Bloom continually improve based on user insights and community-driven ideas, making it an even more powerful tool.

BIBLIOGRAPHY

[1] Objectives
https://colormax.org/color-blind-test/

[2] Study about color blindness app
https://typeset.io/questions/what-are-study-about-color-blindness-app-add-citation-9ioh4zqpsr

[3] The scope of the web application for color-blind individuals
https://www.toptal.com/designers/colorfilter

[4] Ishihara test
https://en.wikipedia.org/wiki/Ishihara_test

[5] Color detection algorithm
https://github.com/balajisrinivas/Color-Detection-OpenCV/blob/main/color_detection.py

[6] OpenCV
https://viso.ai/computer-vision/opencv/

[7] flask
https://flask.palletsprojects.com/en/3.0.x/

APPENDIX A

SAMPLE CODE

1.Backend

```
import cv2
import pandas as pd
from colordata import getColorInfo

# Reading csv file with pandas and giving names to each column
index = [”color”, ”R”, ”G”, ”B”, ”hex”]
csv = pd.read_csv(’./colorshades.csv’, names=index, header=None)

# function to calculate minimum distance from all colors and get the most matching color
def get_color_name(R, G, B):
    minimum = 10000
    for i in range(len(csv)):
        d = abs(R - int(csv.loc[i, ”R”])) + abs(G - int(csv.loc[i, ”G”])) + abs(B - int(csv.
            loc[i, ”B”]))
        if d <= minimum:
            minimum = d
            cname = csv.loc[i, ”color”]
            hex = csv.loc[i, ”hex”]
```

```
    color_info = getColorInfo(cname)
    return cname,hex,color_info


def color_function(img_path,x, y):
    img = cv2.imread(img_path)
    x = int(x)
    y = int(y)
    # Get the BGR values of the 8−adjacent pixels and the center pixel
    pixels = [
        img[y−1, x−1], img[y−1, x], img[y−1, x+1],
        img[y, x−1], img[y, x], img[y, x+1],
        img[y+1, x−1], img[y+1, x], img[y+1, x+1]
    ]

    # Calculate the average BGR values
    b_avg = int(sum([pixel[0] for pixel in pixels]) / 9)
    g_avg = int(sum([pixel[1] for pixel in pixels]) / 9)
    r_avg = int(sum([pixel[2] for pixel in pixels]) / 9)

    # Get the color name based on the average BGR values
    color = get_color_name(r_avg, g_avg, b_avg)
    return color
```

2.Frontend

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF−8">
    <meta name="viewport" content="width=device−width, initial−scale=1.0">
```

```
<title>Bloom: Your Color Guide</title>
<style>
    body {
        font-family: 'Lucida Sans', 'Lucida Sans Regular', 'Lucida Grande', 'Lucida
            Sans Unicode', Geneva, Verdana, sans-serif;
        margin: 0;
        padding: 0;
        height: 40px;
        background-color: #f2ead7;
        color: #744523;
        display: flex;
        flex-direction: column;
        align-items: center;
    }
    header {
        background-color: #744523;
        color: #fff;
        width: 100%;
        height: 40px;
        padding: 20px;
        text-align: center;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
    }
    .container {
        width: 700px;
        margin: 20px auto;
        background: #fff;
        padding: 20px;
```

```
        border-radius: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
        text-align: center;
        position: relative;
    }
    h1 {
        margin: 0;
        font-size: 2em;
    }
    h2 {
        color: #744523;
        margin: 0px 0px 60px 60px;
    }
    .form-container {
        display: flex;
        flex-direction: row;
        align-items: center;
        gap: 10px;
        margin-bottom: 20px;
    }
    input[type="file"] {
        padding: 10px;
        border: 1px solid #ddd;
        border-radius: 4px;
        width: 100%;
        max-width: 300px;
    }
    input[type="submit"] {
```

```
        background–color: #744523;
        color: #fff;
        border: none;
        margin–top: 20px;
        padding: 10px 20px;
        border–radius: 4px;
        cursor: pointer;
        transition: background–color 0.3s;
    }
    input[type=”submit”]:hover {
        background–color: #692d04;
    }
    #uploadedImage {
        margin–top: 20px;
        width: 700px;
        height: 550px;
        border–radius: 8px;
        cursor: crosshair;
        display: block;
    }
    .output–container {
        display: none;
        text–align: center;
        margin–top: 20px;
    }
    #popup {
        display: none;
        position: absolute;
```

```
        top: 10px; /* Adjust this value to position the popup just above the image */
        left: 50%;
        transform: translateX(-50%);
        background: rgba(242, 235, 214, 0.95);
        padding: 15px;
        border-radius: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
        z-index: 10;
        text-align: center;
    }
    #popup h2 {
        margin: 0 0 10px;
    }
    #popup button {
        background-color: #744523;
        color: #fff;
        border: none;
        margin-top: 10px;
        padding: 5px 10px;
        border-radius: 4px;
        cursor: pointer;
        transition: background-color 0.3s;
    }
    #popup button:hover {
        background-color: #692d04;
    }
    #colorInfo {
        background: #f8f8f8;
```

```
        padding: 10px;
        border-radius: 4px;
        margin-top: 10px;
        box-shadow: 0 1px 2px rgba(0, 0, 0, 0.1);
    }
    @media (max-width: 600px) {
        header {
            padding: 15px;
            font-size: small;
        }
        h1 {
            font-size: 1.5em;
        }
        .container {
            padding: 15px;
            width: 90%;
            font-size: small;
        }
        input[type="file"],
        input[type="submit"] {
            width: 100%;
            max-width: none;
        }
        #popup {
            width: 90%;
            padding: 10px;
        }
    }
```

```
    </style>
</head>
<body>
    <header style="background-color: #744523; color: #fff; padding: 15px;">
        <div style="display: flex; justify-content: space-between; align-items: center; max
            -width: 700px; margin: 0 auto;">
            <h1 style="margin: 0; font-size: 1.5em;">Bloom: Your Color Guide</h1>
            <nav>
                <a href="{{url_for('home')}}" style="color: #fff; text-decoration: none;
                    margin-left: 20px;">Home</a>
                <a href="{{url_for('test1')}}" style="color: #fff; text-decoration: none;
                    margin-left: 20px;">Ishihara Test</a>
            </nav>
        </div>
    </header>
    <div class="container">
        <div class="form-container">
            <h2>Upload an Image</h2>
            <form id="uploadForm" action="/upload" method="post" enctype="multipart/
                form-data">
                <input type="file" name="image" accept="image/*" required />
                <input type="submit" value="Upload" />
            </form>
        </div>
        <div class="output-container" id="outputContainer">
            <img id="uploadedImage" src="../{{ image_filename }}" alt="Uploaded Image"
                onclick="getImageCoordinates(event)" width="">
            <div id="popup">
```

```
<h2 id="colorname"></h2>
<button id="showMoreBtn" onclick="toggleDetails()"><svg xmlns="http
    ://www.w3.org/2000/svg" width="16" height="16" fill="currentColor"
    class="bi bi-chevron-double-down" viewBox="0 0 16 16">
     <path fill-rule="evenodd" d="M1.646 6.646a.5.5 0 0 1 .708 0L8
         12.293l5.646-5.647a.5.5 0 0 1 .708.708l-6 6a.5.5 0 0 1-.708 0l
         -6-6a.5.5 0 0 1 0-.708"/>
     <path fill-rule="evenodd" d="M1.646 2.646a.5.5 0 0 1 .708 0L8 8.293
         l5.646-5.647a.5.5 0 0 1 .708.708l-6 6a.5.5 0 0 1-.708 0l-6-6a
         .5.5 0 0 1 0-.708"/>
   </svg>
</button>
<div id="details" style="display: none;">
    <div id="colorInfo"></div>
    <button id="showLessBtn" onclick="toggleDetails()"><svg xmlns="
        http://www.w3.org/2000/svg" width="16" height="16" fill="
        currentColor" class="bi bi-chevron-double-up" viewBox="0 0
        16 16">
         <path fill-rule="evenodd" d="M7.646 2.646a.5.5 0 0 1 .708 0l6 6
             a.5.5 0 0 1-.708.708L8 3.707 2.354 9.354a.5.5 0 1
             1-.708-.708z"/>
         <path fill-rule="evenodd" d="M7.646 6.646a.5.5 0 0 1 .708 0l6 6
             a.5.5 0 0 1-.708.708L8 7.707l-5.646 5.647a.5.5 0 0
             1-.708-.708z"/>
       </svg>
    </button>
</div>
</div>
```

```
        </div>
    </div>
    <script>
        let mobile=0;
        if (window.navigator.userAgent.match(/Android/i)||window.navigator.userAgent.
            match(/iPhone/i)) {
            mobile = 1;
        }
        document.getElementById('uploadForm').addEventListener('submit', function(event
            ) {
            event.preventDefault(); // Prevent the default form submission

            var formData = new FormData(this);
            formData.append('mobile', mobile);
            fetch('/upload', {
                method: 'POST',
                body: formData
            })
            .then(response => response.json())
            .then(data => {
                document.getElementById('uploadedImage').src = `../static/Uploads/${data.
                    image_filename}`;
                if(mobile){
                    document.getElementById('uploadedImage').style.width = '350px';
                    document.getElementById('uploadedImage').style.height = '275px';
                }
                document.getElementById('outputContainer').style.display = 'block';
            })
```

```
        .catch(error => {
            console.error('Error:', error);
        });
    });
    function getImageCoordinates(event) {
        var image = document.getElementById('uploadedImage');
        var rect = image.getBoundingClientRect();
        var x = event.clientX - rect.left;
        var y = event.clientY - rect.top;
        console.log("X Coordinate: " + x + ", Y Coordinate: " + y);
        fetch('/process_coordinates', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ x: x, y: y })
        })
        .then(response => response.json())
        .then(data => {
            document.getElementById('colorname').innerText = "Color: " + data.result
                [0];
            document.getElementById('colorInfo').innerHTML = `
                <p>${data.result[2].description}</p>
                <p><strong>Similar Colors:</strong> ${data.result[2].similar_colors.
                    join(', ')}</p>
                <p><strong>Contrasting Colors:</strong> ${data.result[2].
                    contrasting_colors.join(', ')}</p>
                <p><strong>Hex:</strong> #${ data.result[1]}
```

```
            ';
            var popup = document.getElementById('popup');
            popup.style.display = 'block';
            document.addEventListener('click', hidePopupOnClickOutside);
        })
        .catch(error => console.error('Error:', error));
}
function toggleDetails() {
    var details = document.getElementById('details');
    var showMoreBtn = document.getElementById('showMoreBtn');
    var showLessBtn = document.getElementById('showLessBtn');
    if (details.style.display === 'none') {
        details.style.display = 'block';
        showMoreBtn.style.display = 'none';
    } else {
        details.style.display = 'none';
        showMoreBtn.style.display = 'inline-block';
    }
}
function hidePopupOnClickOutside(event) {
    var image = document.getElementById('uploadedImage');
    var popup = document.getElementById('popup');
    var imageRect = image.getBoundingClientRect();
    var popupRect = popup.getBoundingClientRect();
    if (!isClickInsideRect(event, imageRect) && !isClickInsideRect(event,
        popupRect)) {
        popup.style.display = 'none';
        document.removeEventListener('click', hidePopupOnClickOutside);
```

```
            }
        }
        function isClickInsideRect(event, rect) {
            return event.clientX >= rect.left && event.clientX <= rect.right && event.
                clientY >= rect.top && event.clientY <= rect.bottom;
        }
    </script>
</body>
</html>
```

3.Camera Toggle Button

```
let currentStream = null;
        let front;
        if (window.navigator.userAgent.match(/Android/i)||window.navigator.userAgent.
            match(/iPhone/i)) {
            front = false;
        }else {
            front = true;
        }
        //Camera toggle
        document.getElementById('toggle−btn').onclick = () => {
            front = !front;
            startCamera();
        };
        // Get access to the device camera
        function startCamera() {
            if (currentStream) {
                currentStream.getTracks().forEach(track => track.stop());
            }
```

```
        navigator.mediaDevices.getUserMedia({ video: { facingMode: front ? ”user” : ”
            environment” } })
            .then(function (stream) {
                if(front){
                    video.style.webkitTransform = ”scaleX(−1)”;
                    video.style.transform = ”scaleX(−1)”;
                }else {
                    video.style.webkitTransform = ”none”;
                    video.style.transform = ”none”;
                }
                currentStream = stream;
                videoElement.srcObject = stream;
            })
            .catch(function (err) {
                console.error(’Error accessing the camera:’, err);
            });
    }
```